

Getting started

A lockdown and a vaccination on an SIR model in ten lines

This vignette shows the plain API: build a model, describe interventions in words, inspect them, simulate, and plot. No category theory is needed to use it; the last section says what the theory guarantees behind the scenes.

```
using CategoricalInterventions
using AlgebraicPetri
using DiffEqCallbacks
using LabelledArrays
using OrdinaryDiffEq
using Plots

default(; linewidth=2, grid=false)
```

A model

An AlgebraicPetri SIR net becomes a `Model`. Its transitions become parameter targets and its species state targets. Rates default to the *set-then-scale* algebra (one absolute assignment plus any number of scalings combine; two assignments conflict) and states to the *additive* algebra. Because every transition conserves tokens, a conservation invariant is attached automatically.

```
sir = LabelledPetriNet([:S, :I, :R],
    :inf => ((:S, :I) => (:I, :I)),
    :rec => (:I => :R))

model = Model(sir)
model.invariants, algebra(model.space, Target(:inf))
```

```
(CategoricalInterventions.AbstractInvariant[Conserved([:S, :I, :R])], Affine(Multiplicative(
```

Interventions in words

Each line of an @interventions block names an atom. A lockdown halves the infection rate over [5, 25); a vaccination moves fifty people from S to R at day 15. during= means a span on a parameter, at= an instant on a state.

```
program = @interventions model begin
  lockdown    = scale(:inf, 0.5; during=5..25)
  vaccination = transfer(:S => :R, 50; at=15)
end
```

Program(2 atoms)

explain prints the epochs, the effect on each target, and the pulses.

```
u0 = LVector(S=990.0, I=10.0, R=0.0)
p0 = LVector(inf=0.05 * 10.0 / sum(u0), rec=0.25)

explain(program; baseline=Dict(:inf => p0.inf, :rec => p0.rec))
```

Epochs:

```
[5, 25) lockdown
      inf: 0.0005 → 0.00025  (scale(0.5))
```

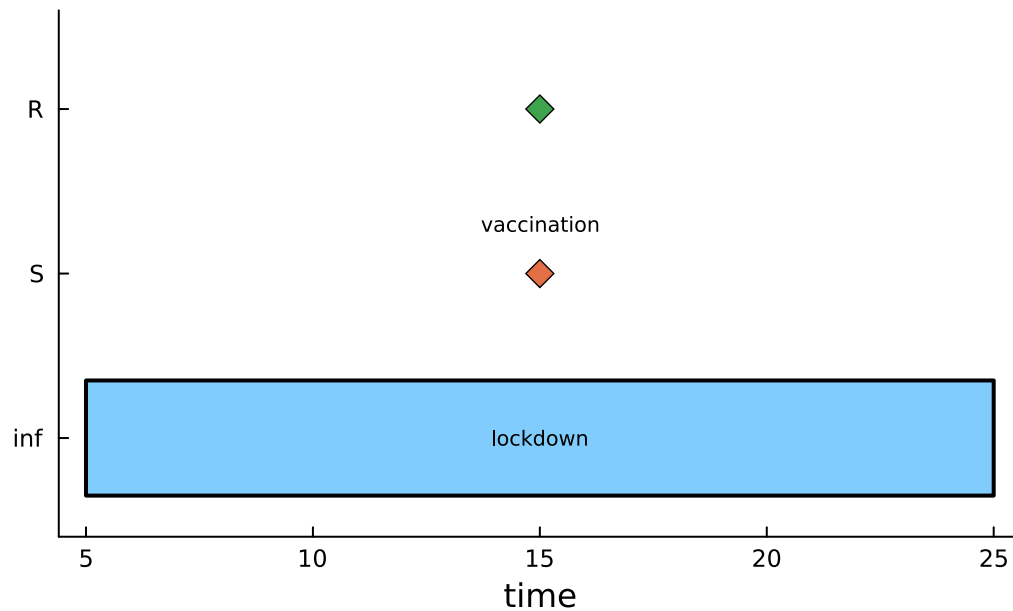
Pulses:

```
at 15  vaccination: S transfer(50 → R)
```

plot(program) draws the same information as a Gantt chart.

```
plot(program; title="Intervention program")
```

Intervention program



Simulate

`simulate` lowers the program to a solver callback, builds the problem from the model's vector field, and solves.

```
tspan = (0.0, 40.0)
solve_with(p) = simulate(model, p; u0, p0, tspan, alg=Tsit5(), saveat=0.5)

baseline      = solve_with(Program(model))
lockdown      = solve_with(Program(model, program[1]))
vaccination   = solve_with(Program(model, program[2]))
combined      = solve_with(program)

(baseline_R = baseline[3, end], lockdown_R = lockdown[3, end],
 vaccination_R = vaccination[3, end], combined_R = combined[3, end],
 combined_total = sum(combined.u[end]))
```

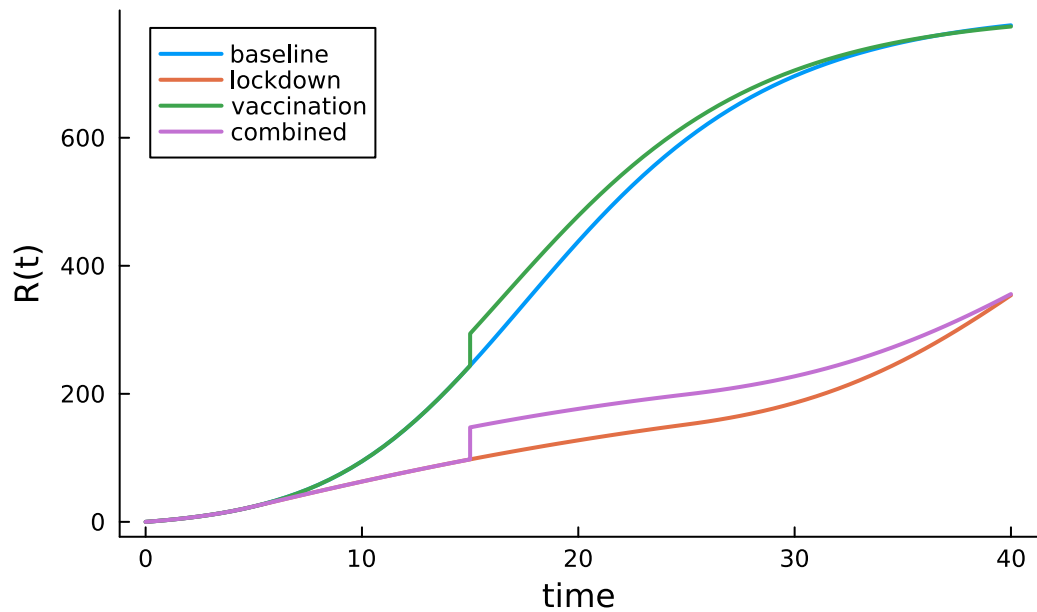
```
(baseline_R = 775.6835919143083, lockdown_R = 354.0601008712396, vaccination_R = 773.72864779)
```

```

plot(baseline.t, baseline[3, :]; label="baseline", xlabel="time", ylabel="R(t)",
     title="Recovered population under intervention scenarios")
plot!(lockdown.t, lockdown[3, :]; label="lockdown")
plot!(vaccination.t, vaccination[3, :]; label="vaccination")
plot!(combined.t, combined[3, :]; label="combined")

```

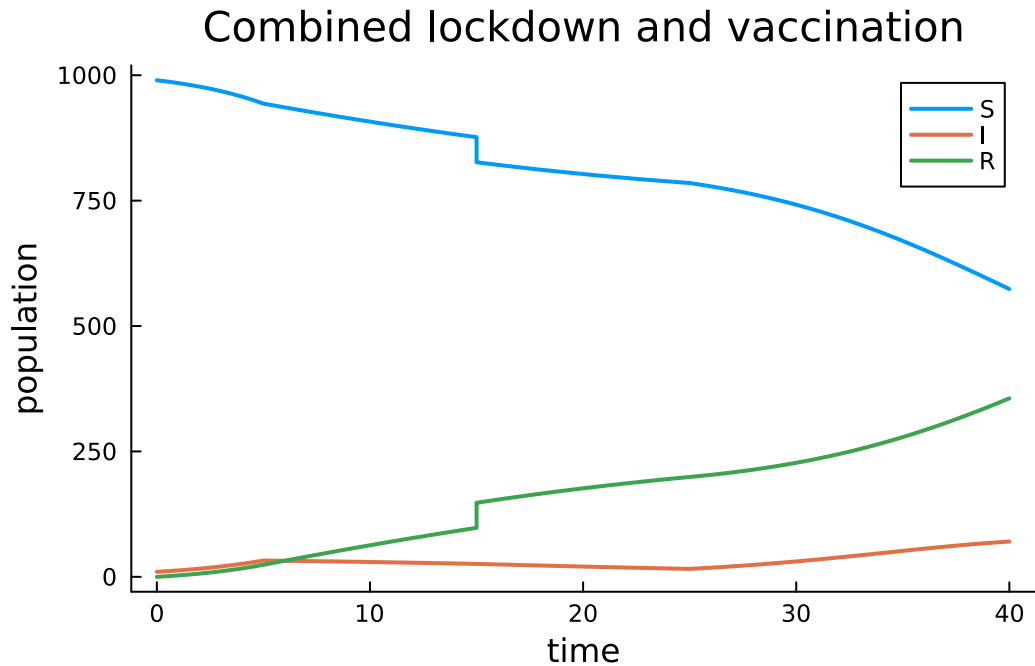
Recovered population under intervention scenari



```

plot(combined.t, combined[1, :]; label="S", xlabel="time", ylabel="population",
     title="Combined lockdown and vaccination")
plot!(combined.t, combined[2, :]; label="I")
plot!(combined.t, combined[3, :]; label="R")

```



What the category theory bought

The two atoms touch different targets, so they compose without any check beyond the type of each target (`different_target_compatible`). The epoch table printed by `explain` is exactly the schedule that the solver callback realises: holding each event's value until the next event reproduces the program's action on the baseline (`lower_eq_apply`). The transfer is one atom that conserves the total population by construction (`transfer_preserves_sum`), which is why the invariant check passes. These theorems are about the algebra and the lowering algorithm; the numerical solution is the solver's business.